

SESSION 12

CRUD Operations

Topics Covered: Create, Read, Update, Delete, Editing using ModelForm, Delete confirmation page

Activities: CRUD for Doctor: Add -> View -> Edit -> Delete

Task 1: Create Product model for Flipkart-style app with fields: name, price, category, then use ModelForm to build form for adding new products.

Explanation & Approach: Sets up model schema and ModelForm for full CRUD workflow.

```
# models.py
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    category = models.CharField(max_length=50)

    def __str__(self):
        return f"{self.name} - ₹{self.price}"

# forms.py
from django import forms

class ProductForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ['name', 'price', 'category']
```

Task 2: Build view in Django displaying list of all products showing name and price in table, with 'Edit' and 'Delete' buttons.

Explanation & Approach: Read operation fetching all records via *Product.objects.all()*.

```
# views.py
from django.shortcuts import render
from .models import Product

def product_list(request):
    products = Product.objects.all()
    return render(request, 'products/list.html', {'products': products})
```

Task 3: Implement edit feature for Product model: clicking 'Edit' opens pre-filled ModelForm allowing user to update details.

Explanation & Approach: Passing *instance=product* pre-populates ModelForm with existing record values for update operations.

```
# views.py
from django.shortcuts import render, get_object_or_404, redirect
from .models import Product
from .forms import ProductForm

def product_edit(request, pk):
    product = get_object_or_404(Product, pk=pk)
    if request.method == 'POST':
        form = ProductForm(request.POST, instance=product)
        if form.is_valid():
            form.save()
            return redirect('product_list')
    else:
        form = ProductForm(instance=product)
    return render(request, 'products/form.html', {'form': form, 'title': 'Edit Product'})
```

Task 4: Add delete confirmation page: clicking 'Delete' shows confirmation page with product name and 'Yes, delete' and 'Cancel' options.

Explanation & Approach: *Two-step delete pattern prevents accidental deletion of database records.*

```
# views.py
def product_delete(request, pk):
    product = get_object_or_404(Product, pk=pk)
    if request.method == 'POST':
        product.delete()
        return redirect('product_list')
    return render(request, 'products/delete_confirm.html', {'product': product})
```